Создание Помощника по Базе Знаний с Agno и Ollama: Глубокое Погружение

Дата анализа: 2026-01-15

В эпоху, когда информация является ключевым активом, способность быстро и точно извлекать знания из внутренних документов становится критически важной. ИИ-агенты, особенно те, что работают с базами знаний, предлагают революционное решение. В этой статье мы тщательно исследуем, как создать такого помощника, используя мощный фреймворк Agno и локально развернутые языковые модели через Ollama, обеспечивая при этом полную конфиденциальность данных.

Часть 1: Agno — Новый Лидер в Мире Агентных Фреймворков

Прежде чем погрузиться в создание помощника, необходимо понять, что представляет собой Agno и почему он привлекает столько внимания разработчиков. Agno — это не просто очередной инструмент, а комплексная экосистема для создания, запуска и управления интеллектуальными агентными системами.

Что такое Agno и его ключевые компоненты?

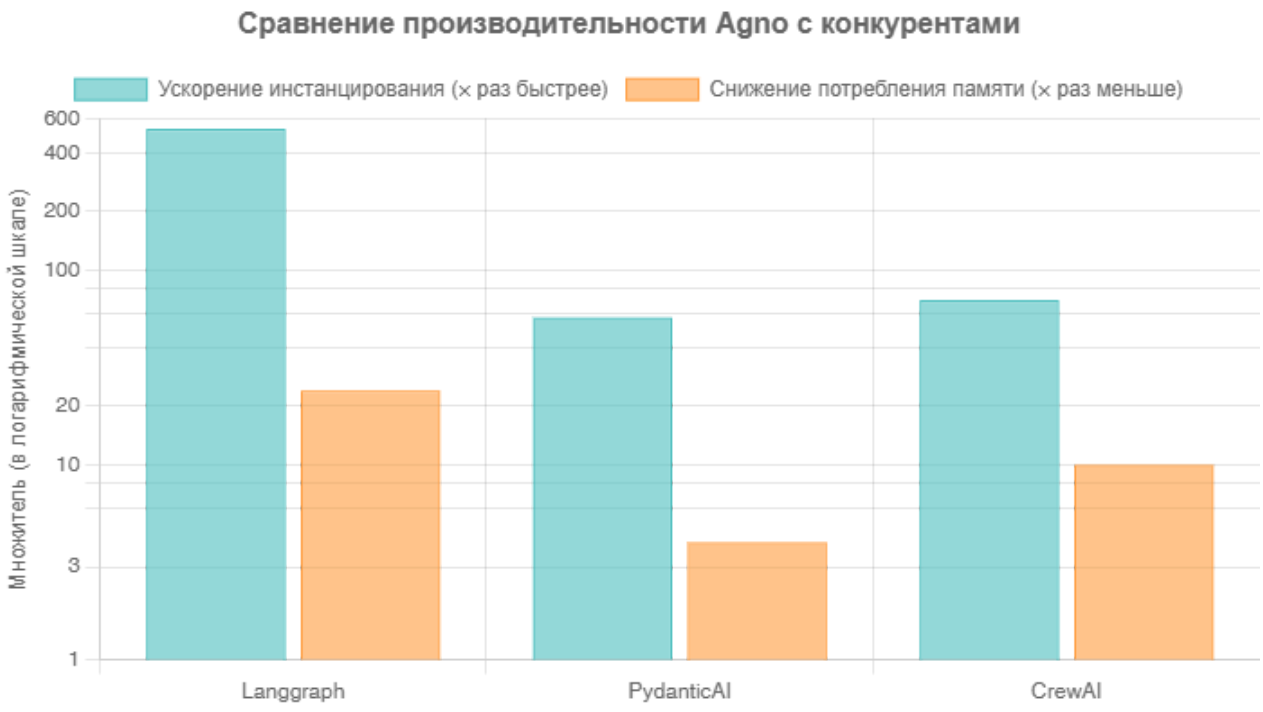
Agno — это высокопроизводительный open-source фреймворк, предназначенный для создания модульных и автономных ИИ-агентов. Ранее известный как Phidata, проект прошел ребрендинг в начале 2025 года, чтобы лучше отразить свою миссию — стать самой быстрой библиотекой для создания мультимодальных агентов Agno is now Generally Available, . Экосистема Agno состоит из трех основных частей:

- **Framework:** Python-библиотека с открытым исходным кодом, которая предоставляет все необходимое для создания агентов, команд и рабочих процессов с памятью, знаниями и инструментами.
- **AgentOS (Runtime):** Высокопроизводительная среда выполнения, которая превращает агентов в масштабируемую инфраструктуру, позволяя запускать их как единый API.
- **Control Plane:** Интуитивно понятный пользовательский интерфейс для мониторинга и управления всей агентной системой в реальном времени.

Эта трехслойная архитектура обеспечивает комплексное решение "под ключ" — от разработки до производственной эксплуатации .

Производительность и философия разработки

Одним из главных преимуществ Agno является его феноменальная производительность и низкое потребление ресурсов. Согласно официальным бенчмаркам, Agno значительно превосходит конкурентов.



Источник: Данные о производительности с agno.com.

Такие показатели достигаются благодаря продуманной архитектуре и фокусу на "чистом Python" подходе, который избегает сложных графов и цепочек, характерных для других фреймворков. Философия Agno — это минимализм и элегантность кода. Разработчики отмечают, что API Agno имеет "меньшую ментальную нагрузку" по сравнению с LangChain , а создание агента для веб-поиска может занять всего 10 строк кода .

Применение Agno в реальных проектах

Сообщество разработчиков активно использует Agno для решения разнообразных задач, что подтверждается многочисленными отзывами. Примеры включают:

- **Автоматизация контента:** Создание видеороликов кинематографического качества из одного промпта, генерация постов для социальных сетей.
- **Персональные ассистенты:** Разработка ИИ-агента "Sous Chef" для планирования семейного питания.
- **Исследовательские агенты:** Системы, которые могут искать информацию в интернете, анализировать лучшие ссылки и составлять отчеты.
- **Интеграции с бизнес-инструментами:** Подключение агентов к Slack, WhatsApp, Telegram для создания интерактивных рабочих процессов.

Один из разработчиков так охарактеризовал свой опыт: "После использования Langgraph... я начинаю новые проекты только с @AgnoAgi, все просто имеет больше смысла, хорошо спроектировано, гибко и намного, намного быстрее" .

Часть 2: Создание Помощника по Базе Знаний с Agno и Ollama

Теперь, когда мы понимаем мощь и гибкость Agno, давайте рассмотрим, как применить его для создания помощника, который будет отвечать на вопросы, используя внутреннюю базу знаний (например, PDF-документы) и локальную языковую модель.

Ключевая концепция: Agentic RAG

Традиционный подход Retrieval-Augmented Generation (RAG) заключается в том, чтобы найти релевантную информацию в векторной базе данных и "вставить" ее в промпт для языковой модели. Это двухэтапный процесс: поиск и дополнение. Однако у этого подхода есть недостаток: контекст добавляется всегда, независимо от того, нужен ли он на самом деле, что может приводить к лишним затратам токенов и не всегда оптимальным ответам.

Agno (еще со времен Phidata) был пионером в подходе, который они называют **Agentic RAG** или Auto-RAG . Суть этого подхода в том, что агент сам принимает решение, нужно ли ему обращаться к базе знаний для ответа на вопрос. Вместо слепого добавления контекста, агенту предоставляется инструмент для поиска по базе знаний. Он анализирует запрос пользователя и решает, использовать ли этот инструмент и с какими параметрами. Это делает агента более "умным", экономным и точным.

В коде это реализуется через простой параметр: `search_knowledge=True` дает агенту инструмент для поиска, в то время как `add_context=True` включает традиционный RAG .

Настройка локальной среды с Ollama

Ollama — это фантастический инструмент, который позволяет запускать большие языковые модели (такие как Llama 3.2, Phi-4, Qwen) прямо на вашем компьютере . Интеграция Agno с Ollama позволяет создать полностью локальную и конфиденциальную систему: все данные, включая документы и запросы пользователей, остаются в вашей среде.

Настройка агента для работы с локальной моделью через Ollama выглядит предельно просто. Вам нужно лишь указать класс модели `Ollama` и ее идентификатор.

```
# Пример конфигурации агента с локальной моделью
from agno.agent import Agent
from agno.models.ollama import Ollama

# Предполагается, что модель 'llama3.2' запущена локально через 'ollama run llama3.2'
local_llm_agent = Agent(
    model=Ollama(id="llama3.2"),
    instructions="Ты — полезный ассистент."
```

```
)

# Запуск агента
# local_llm_agent.print_response("Объясни, что такое ИИ-агент простыми словами.")
```

Agno автоматически подключается к серверу Ollama, который по умолчанию работает на `http://localhost:11434`.

Пошаговое руководство по созданию помощника (концептуально)

Создание помощника по базе знаний с использованием Agentic RAG включает три основных шага:

1. **Определение Базы Знаний:** Сначала необходимо создать объект `Knowledge`. Он отвечает за хранение и индексацию информации. В качестве хранилища можно использовать векторные базы данных, такие как PgVector или LanceDB. Информация из источников (например, PDF-файлов) разбивается на части (чанки), преобразуется в векторы с помощью эмбеддера (например, `OllamaEmbedder`) и сохраняется.

```
from agno.knowledge import Knowledge
from agno.knowledge.document import PDFKnowledge
from agno.vectordb.pgvector import PgVector
from agno.embedder.ollama import OllamaEmbedder

# Настройка векторной базы данных
db_url = "postgresql+psycopg://ai:ai@localhost:5532/ai"
vector_db = PgVector(collection="company_docs", db_url=db_url)

# Настройка эмбеддера для создания векторов локально
local_embedder = OllamaEmbedder(model="nomic-embed-text")

# Создание базы знаний из PDF-документов
knowledge_base = Knowledge(
    sources=[
        PDFKnowledge(path="path/to/your/pdfs/")
    ],
    vector_db=vector_db,
    embedder=local_embedder
)

# Загрузка и индексация документов (этот процесс запускается один раз)
# knowledge_base.load()
```

2. **Создание Агента:** Далее мы создаем экземпляр `Agent`, передавая ему нашу базу знаний и включая режим Agentic RAG.

```
from agno.agent import Agent
from agno.models.ollama import Ollama

knowledge_assistant = Agent(
    model=Ollama(id="llama3.2"),
    knowledge=knowledge_base,
    search_knowledge=True,  # Включаем Agentic RAG
    instructions=[
```

```
            "Ты — эксперт по внутренним документам компании.",
            "Отвечай на вопросы, используя предоставленную базу знаний.",
            "Если информация отсутствует, сообщи об этом."
        ]
    )
```

3. **Запуск и взаимодействие:** Теперь агент готов к работе. Можно отправлять ему запросы, и он будет самостоятельно решать, нужно ли ему искать информацию в документах для формирования ответа.

```
# Пример взаимодействия с агентом в режиме разработки
knowledge_assistant.print_response(
    "Каковы ключевые положения политики безопасности, описанные в документе X?"
)
```

Этот подход не только обеспечивает высокую точность ответов, но и дает полную прозрачность процесса принятия решений агентом, если включить режим отладки `debug_mode=True`), который показывает его "мысли" и вызовы инструментов .

Часть 3: Сравнение и Продвинутые Концепции

Хотя Agno предлагает простой и быстрый старт, важно понимать его место в экосистеме ИИ-инструментов и его возможности для масштабирования.

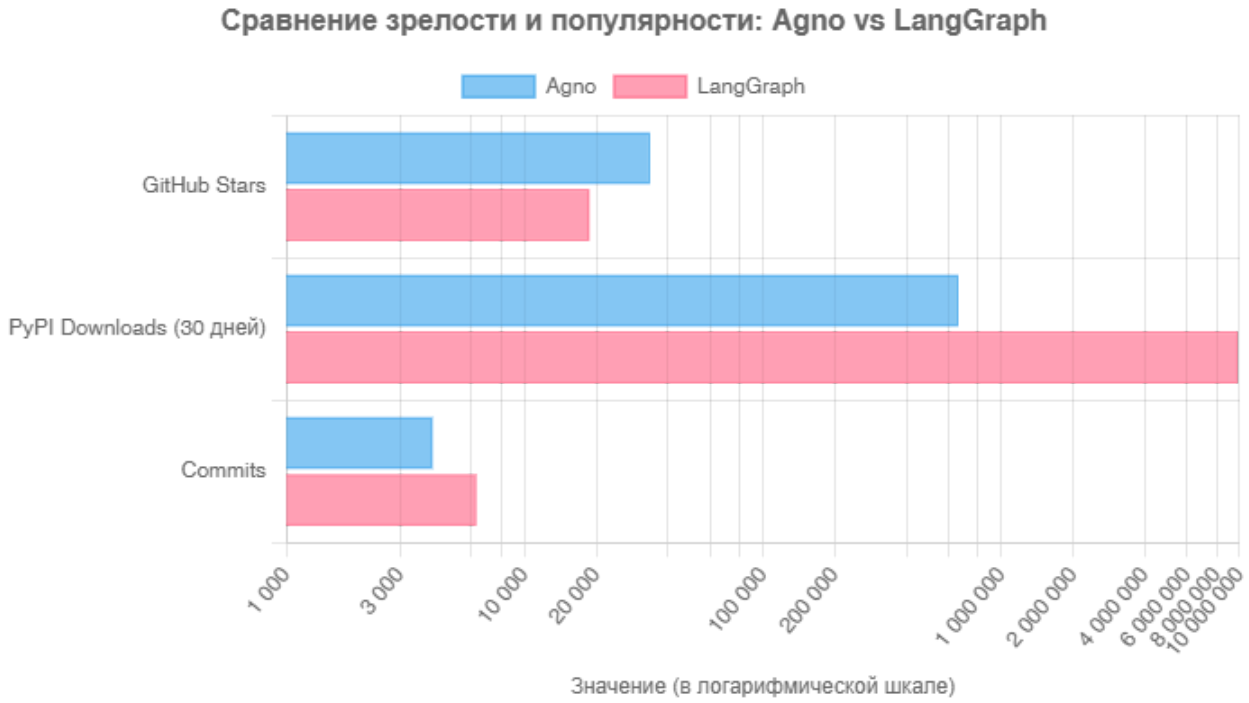
Agno в сравнении с LangChain/LangGraph

Выбор между Agno и LangChain/LangGraph часто зависит от целей проекта и стиля разработки. Оба фреймворка мощные, но имеют разные философии.

- **Философия:** LangChain — это модульный "ящик с инструментами";, предоставляющий максимальную гибкость и контроль. LangGraph расширяет его, позволяя строить сложные, циклические рабочие процессы в виде графов состояний. Agno, напротив, предлагает более высокоуровневые абстракции `Agent`, `Team`) и оптимизирован для скорости разработки и производительности, следуя "конфигурационному" подходу .

- **Кривая обучения:** Agno считается более дружелюбным для новичков благодаря своему чистому Python API. LangGraph требует понимания концепций графов и машин состояний, что может быть сложнее для освоения.

- **Сообщество и зрелость:** LangGraph, будучи частью экосистемы LangChain, имеет огромное количество загрузок на PyPI, что говорит о широком распространении. Agno, хотя и моложе, демонстрирует взрывной рост интереса в сообществе, что отражается в большем количестве звезд на GitHub.



Источник: Данные сравнения из статьи Agno vs LangGraph: Best Framework to Build Multi-Agent... (данные на сентябрь 2025 г.).

Итог: Выбирайте LangChain/LangGraph, если вам нужен полный контроль, сложная кастомная логика и вы готовы инвестировать время в разработку. Выбирайте Agno, если ваша цель — быстро запустить надежных, ориентированных на конкретные задачи агентов с минимальными усилиями .

За пределами одного агента: Команды и Рабочие процессы

Настоящая сила Agno раскрывается при создании мультиагентных систем. Вместо того чтобы создавать одного монолитного агента, вы можете построить "команду" из нескольких специализированных агентов, которые сотрудничают для решения сложной задачи.

Например, нашего помощника по базе знаний можно расширить до исследовательской команды:

- **Агент-аналитик (Knowledge Agent):** Основной агент, который ищет информацию во внутренней базе знаний.
- **Веб-исследователь (Web Researcher):** Агент, оснащенный инструментами для поиска в интернете (например, `DuckDuckGoTools`), который ищет самую свежую информацию по теме.
- **Агент-синтезатор (Synthesizer Agent):** Агент, который получает данные от первых двух, анализирует их, разрешает противоречия и составляет итоговый, полный отчет.

Agno упрощает организацию такого взаимодействия с помощью абстракции `Team`, которая поддерживает различные режимы сотрудничества, такие как `coordinate` (один агент-лидер делегирует задачи) или `collaborate` (агенты общаются на равных) . Это позволяет создавать

сложные, но управляемые системы, где каждый агент отвечает за свою узкую область, повышая общую эффективность и надежность.

Заключение

На текущий момент, 2026-01-15, Agno представляет собой один из самых перспективных и мощных фреймворков для создания ИИ-агентов. Его фокус на производительности, простоте использования и комплексном подходе от разработки до эксплуатации делает его идеальным выбором для широкого круга задач.

Создание помощника по базе знаний с использованием Agno и локальных моделей через Ollama — это не только практичное, но и стратегически верное решение. Оно обеспечивает полную конфиденциальность данных, высокую скорость работы и интеллектуальное поведение благодаря подходу Agentic RAG. Если вы ищете быстрый, надежный и масштабируемый способ внедрения агентных технологий в свои проекты, Agno, несомненно, заслуживает вашего самого пристального внимания.

Reference

[1] Идеи проектов для AI Agents : r/LLMDevs
https://www.reddit.com/r/LLMDevs/comments/1joqiv0/project_ideas_for_ai_agents/?tl=ru

[2] LangChain vs Agno: How to Choose the Right AI Agent ...
https://bix-tech.com/langchain-vs-agno-how-to-choose-the-right-ai-agent-framework-for-your-use-case/

[3] Agno vs LangGraph: Best Framework to Build Multi-Agent ...
https://www.zenml.io/blog/agno-vs-langgraph

[4] Phidata is now Agno. Announcing our new brand identity ...
https://www.linkedin.com/posts/ashpreetbedi_phidata-is-now-agno-announcing-our-new-activity-7290750031312179200-mGAd

[5] Agno: Agent Framework and High-Performance Runtime for ...
https://www.agno.com/

[6] Agno: Agent framework 10000x faster than LangChain
https://news.ycombinator.com/item?id=43274435

[7] agno-agi/phidata
https://github.com/agno-agi/phidata

[8] Examples - Phidata
https://docs.phidata.com/more-examples

[9] Knowledge - Phidata
https://docs.phidata.com/agents/knowledge

[10] Ollama - Phidata - Agno
https://docs.phidata.com/models/ollama

[11] Ollama
https://docs.agno.com/integrations/models/local/ollama/overview

[12] Understanding Agno: A Fast, Scalable Multi-Agent ...
https://www.digitalocean.com/community/conceptual-articles/agno-fast-scalable-multi-agent-framework

[13] Agno is now Generally Available

https://www.agno.com/blog/ga